

## CAPSTONE PROGRESS REPORT

# ALMS

## Agentic Legacy Modernization System

*An 8-week account of designing, building, and evaluating a fully local, multi-agent pipeline that migrates Python monoliths into deployable FastAPI microservices — at zero API cost.*

---

B.Tech Capstone – Computer Science & Business Systems  
Progress Report · August 2026

## What This Report Covers

This report documents eight weeks of work on ALMS, a system for automating monolith-to-microservice migration using local, open-weight language models rather than hosted APIs.

ALMS takes a legacy Python monolith and produces a running, containerised microservice stack: it analyzes the codebase's dependency structure, proposes service boundaries, generates FastAPI code and test suites for each proposed service, and packages the result behind a Docker Compose & Nginx gateway. Every step in this pipeline runs entirely on local hardware — there is no cloud inference, no GPU requirement, and no proprietary source code ever leaves the developer's machine.

Over the eight weeks covered here, the project moved from a literature review and mentor-approved roadmap through a working multi-agent architecture (LangGraph-orchestrated Analyzer, Architect, Refactoring, and Test-Generation agents grounded in a local RAG knowledge base) to a full end-to-end run on a real Flask monolith. That run compiled 4 of 4 proposed services cleanly, generated 1,120 test cases across 570 files, reduced the dependency graph used for boundary selection by more than half, and booted 3 of 4 resulting containers behind a working API gateway — using a 30-billion-parameter local model with no cloud cost. The report is equally direct about what did not work: one service never booted, generated tests are not yet compile-gated, and two services ended up duplicating each other's data logic. Each of these gaps has a concrete next step attached, detailed in the closing section.

## Why Migration Needs Automating

### **Boundary identification is the hardest part**

Deciding where to split a monolith is repeatedly flagged as the single hardest sub-problem in the literature.

### **Migrations take months, not days**

Teams manually trace coupling, redesign data ownership, and rewrite request handling by hand.

### **Existing AI tools assume a GPU + cloud budget**

Fine-tuned embedding methods need GPU training; agentic pipelines call hosted models like Claude/Cursor per step.

### **Sending proprietary code to a third-party API isn't always an option**

Students, small teams, and regulated organizations often simply cannot use cloud-hosted LLMs on their codebase.

# What ALMS Does & How It Works

A multi-agent pipeline that takes a legacy Python monolith and produces a running, containerised microservice stack — five stages, one continuous flow.

| Stage        | What Happens                                                             |
|--------------|--------------------------------------------------------------------------|
| 1 • Analyze  | Parse the monolith's AST into a call graph.                              |
| 2 • Design   | Cluster the graph into service boundaries (Louvain community detection). |
| 3 • Generate | Write FastAPI code for each proposed service.                            |
| 4 • Test     | Generate pytest suites automatically for each service.                   |
| 5 • Deploy   | Package as Docker containers behind an Nginx gateway.                    |

**Pipeline detail:** Legacy Monolith → Analyzer (AST + graph) → Architect (Louvain) → Refactor + py\_compile → Test-Gen (pytest) → Deployable Stack, with a retry loop on compile failure (max 3 attempts). Human-in-the-loop review gates sit after Analysis, after Boundary Proposal, and after Test Generation — a reviewer can approve, reject with feedback, or stop. The whole pipeline is orchestrated with LangGraph: a stateful graph fans out one parallel branch per proposed service, each running its own compile-retry loop, then merges results back with a name-keyed reducer. Every LLM stage is grounded in a local knowledge base (ChromaDB + nomic-embed-text) of FastAPI, DDD, refactoring, and security patterns — compensating for what a small local model doesn't reliably know.

# Keep Everything Deterministic Out of the LLM

The model is only asked to do the one transformation it's actually good at. Everything else — boundaries, validation, data contracts — is handled by ordinary, checkable code.

## **Deterministic — no model in the loop**

- AST-based dependency graph extraction
- Filtering out builtin / stdlib noise before clustering
- Louvain community detection (seed-fixed → reproducible)
- py\_compile gate with bounded auto-retry on generated code
- Pydantic schema validation of every inter-agent message

## **LLM — only where it's genuinely needed**

- Writing the FastAPI service code from source modules
- Generating pytest test suites for each service
- Optionally refining bounded-context descriptions
- Grounded throughout by a local retrieval (RAG) knowledge base
- No model is ever used to judge another model's output

# Technology Stack

| Component            | Role                                           |
|----------------------|------------------------------------------------|
| LangGraph            | Stateful orchestration, fan-out & retry cycles |
| Ollama + Qwen        | Local LLM runtime — no cloud inference         |
| ChromaDB             | Local vector store for RAG retrieval           |
| python-louvain       | Community detection for service boundaries     |
| Docker Compose       | Containerised, portable deployment             |
| pytest<br>hypothesis | + Generated & property-based testing           |

## WHY IT MATTERS

# Fully Local — No Cost, No Cloud, No Compromise

**\$0**

API cost — every run is free

**100%**

Offline — no network call at any stage

**0**

GPUs required — runs on a CPU-only laptop

**0**

Lines of source code ever leave the machine

Prior automated pipelines pick one: reach a deployable result by depending on hosted, commercial-scale models (Cursor, Claude Code) and cloud inference — or stay local and stop at a class-to-service partition, never producing a running artifact. ALMS combines a full decompose → generate → test → deploy pipeline with a fully local, offline execution model — a combination no prior system in our review offers.

## RESULTS

# Case Study: A 1,578-Line Flask Monolith

Full pipeline run — analysis through containerised deployment — on a 6-module e-commerce Flask app (users, orders, payments).

**4 of 4**

proposed services  
compiled cleanly (0  
needing manual fixes)

**1,120**

test cases auto-  
generated across 570  
files

**56% /**

**61%**

reduction in graph  
nodes / edges from  
deterministic filtering

**3 of 4**

containers booted  
successfully behind a  
working API gateway

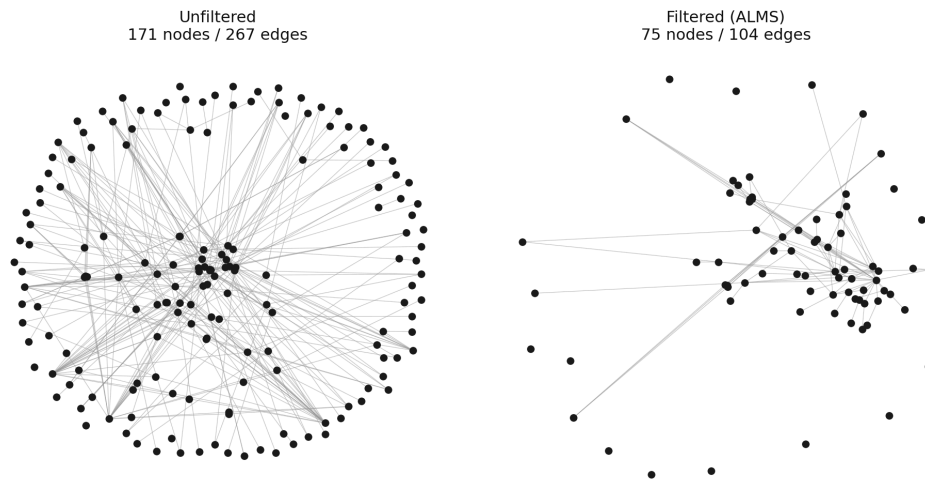
Evaluated with **qwen3-coder:30b** (Mixture-of-Experts, ~3.3B active parameters/token) served locally via Ollama, with weights larger than host RAM paged from disk — a single unattended run completed end-to-end in one pass, at zero API cost.

*Feasibility study on a single codebase — not a benchmark claim. See Limitations, next page.*

## RESULTS

# Deterministic Filtering Before Any Model Sees the Graph

Removing calls to Python builtins, stdlib methods, and local variables — before Louvain clustering ever runs — keeps the boundary-selection graph small and architecturally meaningful.



*Unfiltered dependency graph (171 nodes / 267 edges) vs. ALMS's filtered graph (75 nodes / 104 edges).*

**171 → 75**

nodes (−56.1%)

**267 → 104**

edges (−61.0%)

**6 → 2**

coupling hotspots

# The 8-Week Build

| Week | Milestone            | Details                                                                                                                                     |
|------|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Topic finalized      | Literature review conducted; scope and feasibility of an agentic, local-model-driven migration tool confirmed.                              |
| 2    | Mentor approval      | Project roadmap drafted and presented; mentor consultation completed; required resources procured.                                          |
| 3    | Dev setup            | Development environment and repository structure established; overall multi-agent architecture defined; candidate local models shortlisted. |
| 4    | Model chosen         | Qwen2.5-Coder 7B selected as the initial local model; agent prompt engineering begun and drafted.                                           |
| 5    | Analyzer + Architect | Analyzer and Architect agents implemented; migration methodology reviewed against the literature.                                           |
| 6    | Refactoring agent    | Code-generation (Refactoring) agent completed; RAG knowledge base construction initiated.                                                   |
| 7    | Full integration     | Test-generation agent built; LangGraph/LangChain wiring completed; first full end-to-end run executed (non-optimal results).                |
| 8    | Scaled up            | Performance limitations from Week 7 addressed via a "layering" methodology, enabling an upgrade from the 7B model to Qwen3-Coder 30B.       |

## What We've Built So Far

- Analyzer agent — AST parsing & dependency graph
- Refactoring agent — FastAPI code generation
- RAG knowledge base — ChromaDB + local embeddings
- py\_compile validation gate with bounded retry
- Full end-to-end run completed on a sample monolith
- Architect agent — Louvain-based boundary proposal
- Test-Gen agent — pytest + hypothesis suites
- LangGraph orchestration — fan-out, retry, HITL gates
- Docker Compose deployment + Nginx API gateway

## Where It Fell Short — and Why That's Useful

### **One service never booted**

Application Core Service — leftover routing/glue code with no owned data — got clustered into its own "service" anyway. A hub module with no domain needs a different rule.

### **Test generation isn't compile-gated**

Only 58.2% of the 570 generated test files are valid Python (truncated output). Code generation, which does have a compile-retry loop, passes 100%.

### **Services duplicate each other's data**

Order and Payment services both regenerated /users and /products CRUD logic — declared ownership isn't yet enforced at generation time.

*Each of these directly motivates a concrete next step, below.*

## Conclusion & Future Work

A disciplined deterministic/LLM split lets a local model — even a 30B-class one — carry a full migration pipeline through to a mostly-bootable, containerised output, at zero cloud cost, on hardware a single developer already owns.

- Extend the `py_compile`-style retry gate to Test-Gen output
- Enforce declared data ownership to stop cross-service duplication
- Scale evaluation to 10–30 repositories with ablation studies
- Wire shadow-testing to the live container stack for real parity checks